

Árvores Balanceadas: AVL

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- Um aspecto fundamental do estudo de **árvores binárias de busca** é, naturalmente, o custo de acesso a uma chave desejada
- Com o intuito de minimizar esse custo, foram desenvolvidas as árvores binárias de busca **ótimas**

Introdução (Cont.)

- Porém, se restringe a aplicações estáticas
 - Após inserções/remoções árvore deixar de ser **ótima**
- Para estruturas em que as probabilidades de acesso são idênticas entre si, há uma alternativa, considerada nesta aula

Estrutura Balanceada

- Ideia é manter custo de acesso na mesma ordem de grandeza de uma **árvore ótima**, ou seja, $O(\log n)$
- Custo deve se manter ao longo de toda utilização da estrutura
→ Inclusive após inclusões e remoções
- Para alcançar essa finalidade, a estrutura deve ser alterada, periodicamente, de forma a se moldar aos novos dados
 - Custo dessas alterações é $O(\log n)$
- Uma estrutura que opera com essas características é denominada **balanceada**

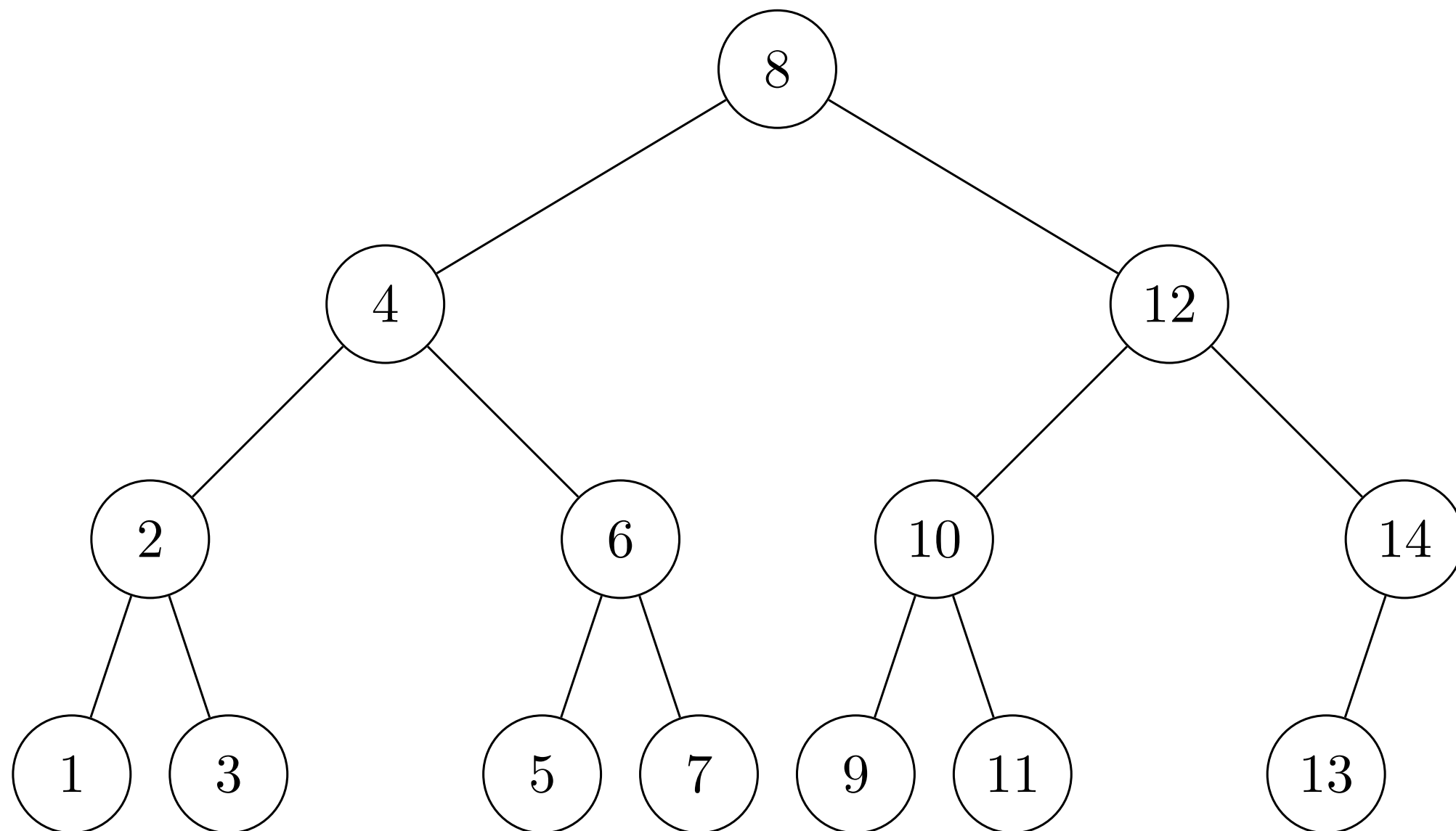
Árvores Completas

- **Árvores completas** são aquelas que minimizam o número de comparações efetuadas no pior caso para uma busca
 - Para uma busca com chaves de probabilidade de ocorrência idênticas
- Do ponto de vista de aplicações dinâmicas, contudo, o uso de **árvores completas** é, em geral desaconselhável
 - Após certo número de inclusões/exclusões a árvore pode assumir uma forma pouco recomendável
 - Em um caso extremo, ela pode inclusive degenerar-se em uma lista

Balanceamento

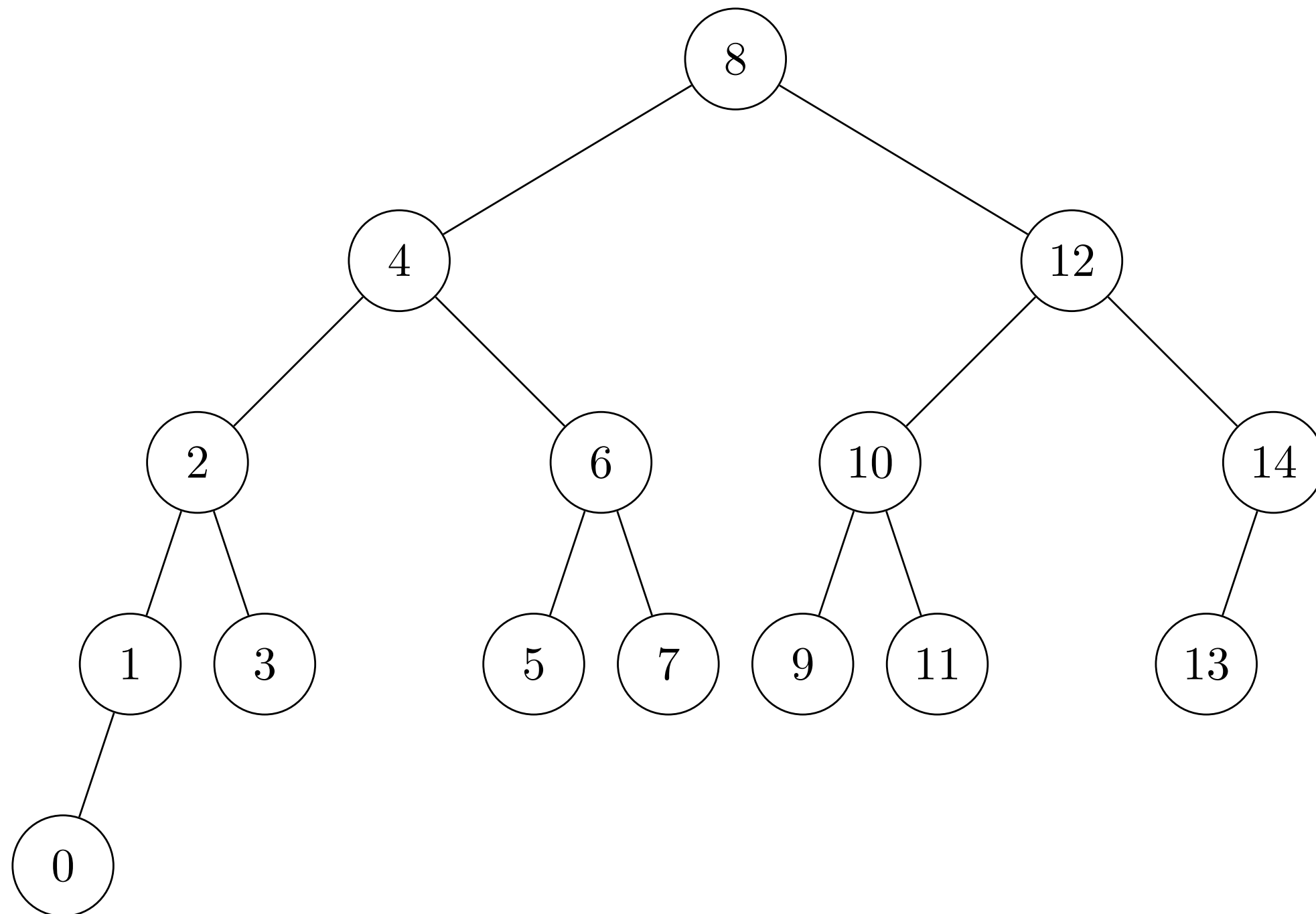
- Uma ideia seria aplicar um algoritmo que tornasse a árvore novamente completa, tão logo tal característica fosse perdida
 - Após uma inclusão ou exclusão
- Como efetuar essa operação utilizando menos que $\Omega(n)$ passos, no pior caso?

Figura: Árvore Completa (a)



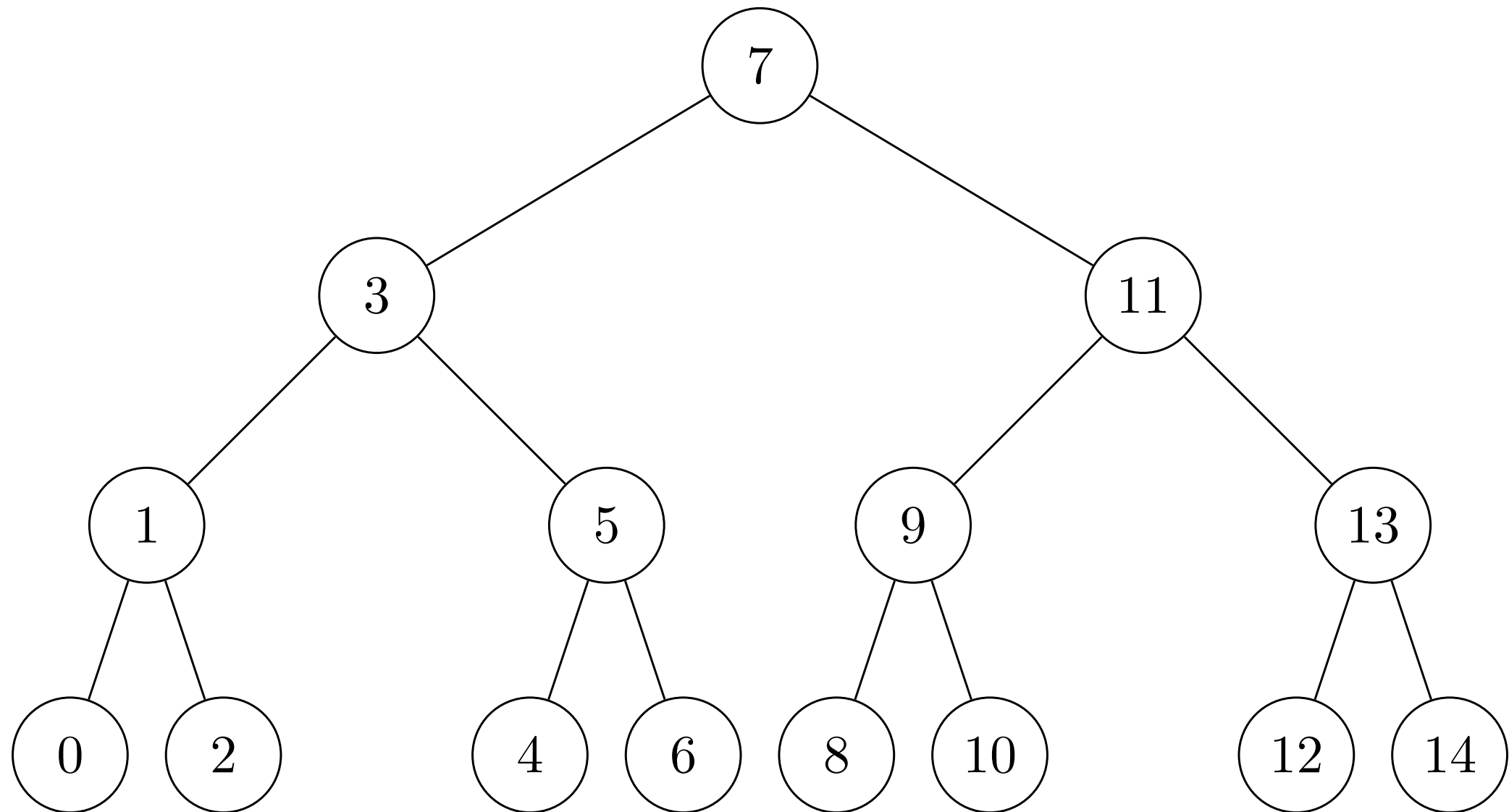
Nessa árvore só há uma possibilidade de inclusão sem aumento de altura. É a inclusão de uma chave maior do que todas as demais.

Figura: Árvore Completa (b)



Se nessa árvore, ao contrário, for incluído um nó de chave menor do que todas as outras, ela deixaria de ser completa.

Figura: Árvore Completa (c)



Para transformar esta última em uma árvore completa, observe que é necessário alterar a posição na estrutura de todos os seus nós.

Restabelecimento Árvore Completa

- O algoritmo de restabelecimento da estrutura requer, pelo menos, **$\Omega(n)$ passos**
- **Custo $\Omega(n)$** é excessivo
 - Normalmente operações como inserção ou remoção seriam efetuados em **$O(\log n)$ passos**
 - Por esse motivo, as árvores completas (e a busca binária) não são recomendadas para aplicações que requeiram estruturas dinâmicas

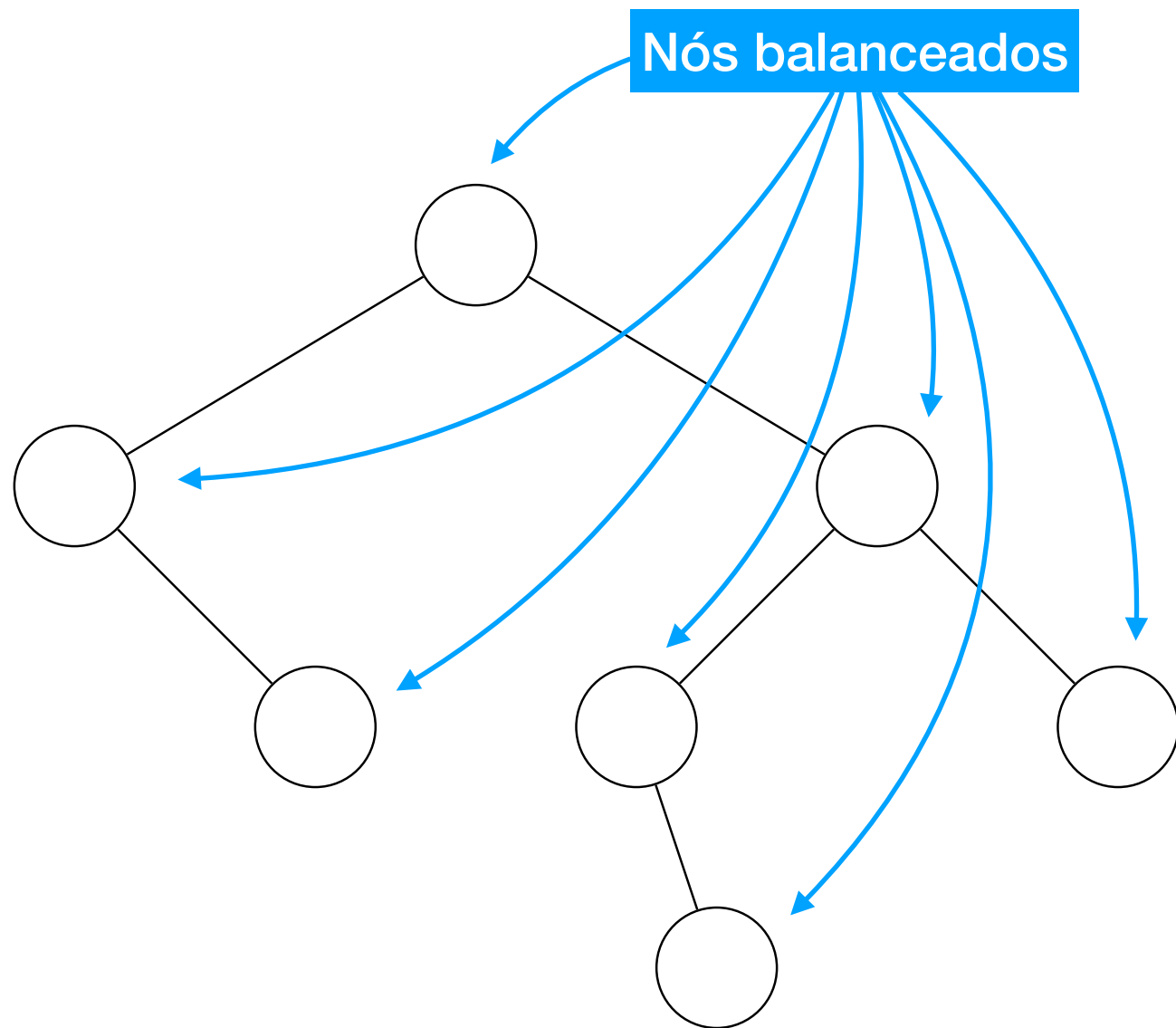
Alternativa

- Uma alternativa é permitir uma **árvore binária** cujo **pior caso** para a busca não seja necessariamente tão pequeno quanto o mínimo $1 + \lfloor \log_2 n \rfloor$ passos alcançado pela **árvore completa**
- Possua altura igual a $O(\log n)$
 - Mesma ordem de grandeza de uma árvore completa
 - Esta propriedade se estende para todas as subárvores
- Uma árvore que satisfaça essa condição é denominada **balanceada**

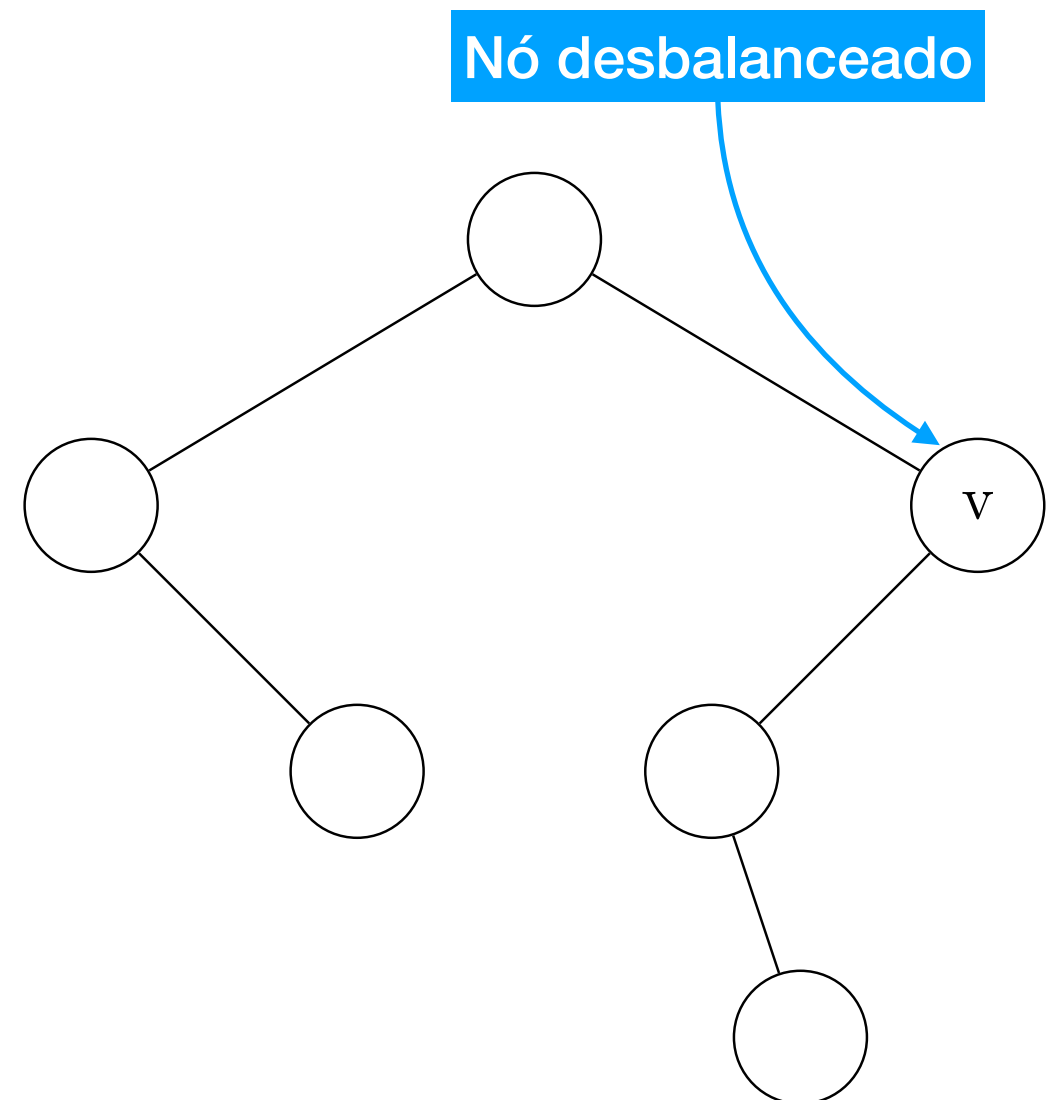
Árvores AVL

- Uma árvore binária T é denominada **AVL** quando, para qualquer nó de T , as alturas de suas duas subárvores, esquerda e direita, diferem em módulo de até uma unidade
- Nesse caso, v é um nó **balanceado (regulado)**
 - Um nó que não satisfaça essa condição de altura é denominado **desbalanceado (desregulado)**

Figura: Árvore AVL e não AVL



(a) Árvore AVL



(b) Árvore não AVL

Inclusão em Árvores AVL

- Seja T uma árvore **AVL**, na qual serão efetuadas inclusões de nós
- Para que T se mantenha como árvore **AVL** após as inclusões (e, se **mantenha balanceada**), é preciso efetuar operações de restabelecimento de regulagem de seus nós

Inclusão em Árvores AVL (Cont.)

- Verifica-se, após cada inclusão, se algum nó p se encontra **desbalanceado**, isto é,
 $\Rightarrow$ se a diferença de altura entre as duas subárvores de p tornou-se maior do que um
- Em caso positivo, aplicar transformações apropriadas para regulá-lo

Transformações

- Serão utilizadas quatro transformações:
 - Rotação direita (*LL Left-Left*)
 - Rotação esquerda (*RR Right-Right*)
 - Rotação dupla direita (*LR Left-Right*)
 - Rotação dupla esquerda (*RL Right-Left*)

Figura: Rotação Direita



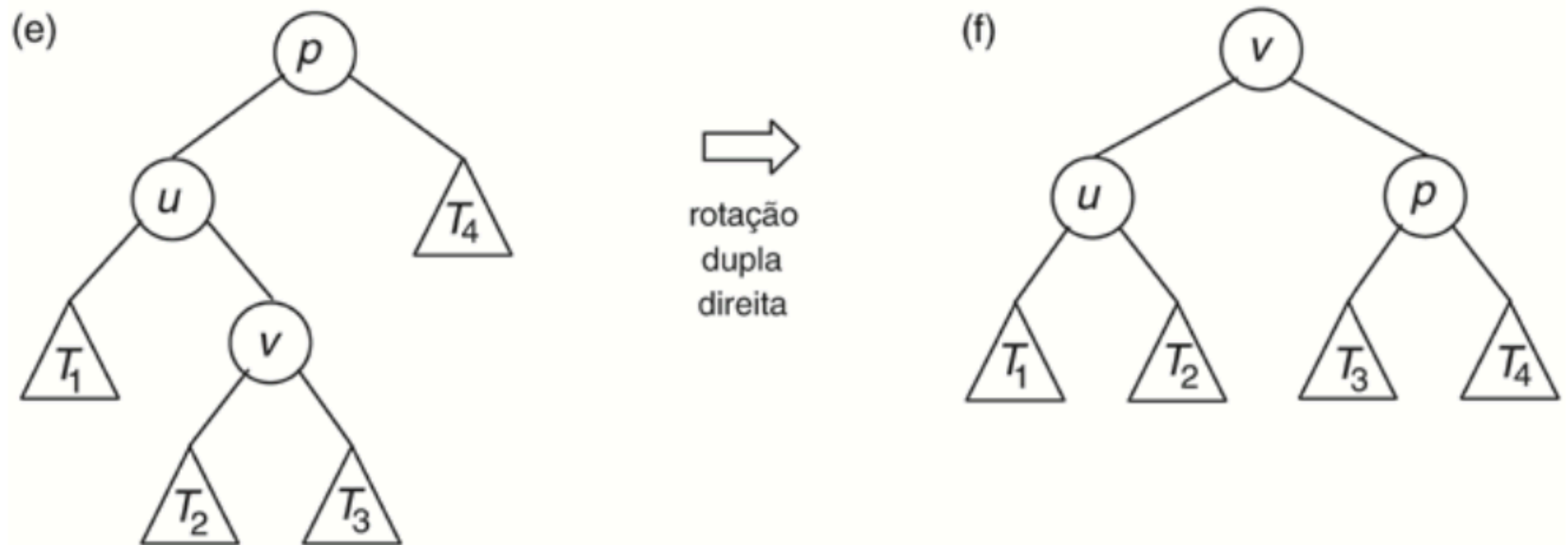
(LL Left-Left) Quando ocorre: Inserção no filho esquerdo do filho esquerdo. Interpretação: A árvore está “pesada à esquerda da esquerda”.

Figura: Rotação Esquerda



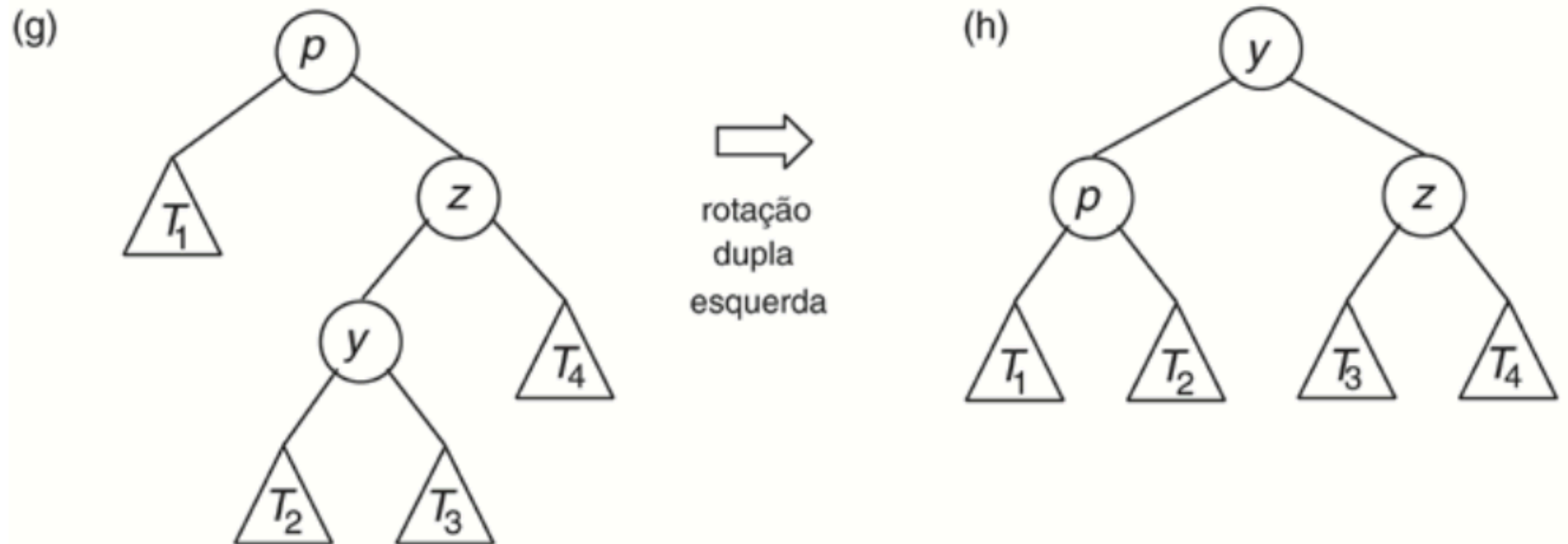
(RR Right-Right) Quando ocorre: Inserção no filho direito do filho direito. Interpretação: A árvore está “pesada à direita da direita”.

Figura: Rotação Dupla Direita



(LR Left-Right) Quando ocorre: Inserção no filho direito do filho esquerdo. Interpretação: A árvore está “esquerda, mas com inclinação para a direita”. Sequência de operações: (i) rotação à esquerda no filho; (ii) rotação à direita no nó.

Figura: Rotação Dupla Esquerda



(RL Right-Left) Quando ocorre: Inserção no filho esquerdo do filho direito. Interpretação: A árvore está “direita, mas com inclinação para a esquerda”. Sequência de operações: (i) rotação à direita no filho; (ii) rotação à esquerda no nó.

Transformações

- As subárvores T_1 , T_2 , T_3 e T_4 que aparecem nas figuras podem ser vazias ou não
- O nó p é chamado **raiz** da transformação
- Essas transformações preservam a natureza da árvore como sendo **binária de busca**

Análise Inclusão

- Suponha que um nó q foi incluído em T
- Se após a inclusão todos os nós se mantiveram **balanceados** então
 - A árvore manteve-se **AVL** e não há nada a efetuar
- Caso contrário:
 - Seja p o nó mais próximo às folhas de T que se tornou **desbalanceado**
 - Não há ambiguidade na escolha de p , pois qualquer subárvore de T que se tornou **desbalanceada** após a inclusão de q deve necessariamente conter p
 - Então p se encontra no caminho de q à raiz de T , e sua escolha é única

Análise Inclusão (Cont.)

- Sejam $h_E(p)$ e $h_D(p)$ as **alturas das subárvores** esquerda e direita de p , respectivamente
- Naturalmente, $|h_E(p) - h_D(p)| > 1$
- Então, conclui-se que $|h_E(p) - h_D(p)| = 2$, pois T era uma árvore **AVL**, e, além disso, a inclusão de um nó não pode aumentar em mais de uma unidade a altura de qualquer subárvore
- Podem-se identificar vários casos

Caso 1: $h_E(p) > h_D(p)$

- Então, q pertence à subárvore esquerda de p .

Veja, se anteriormente a árvore era balanceada, então existem três possibilidades:

- $h_E(p) < h_D(p)$ e $h_D(p) - h_E(p) = +1$
- $h_E(p) = h_D(p)$ e $h_D(p) - h_E(p) = 0$
- $h_E(p) > h_D(p)$ e $h_D(p) - h_E(p) = -1$

A única possibilidade no qual $h_E(p) > h_D(p)$ e a árvore se torna desbalanceada, é se um nó q é adicionado à subárvore esquerda de p .

Caso 1: $h_E(p) > h_D(p)$ (2)

- Então, q pertence à subárvore esquerda de p . Além disso, p possui o filho esquerdo $u \neq q$.

Se $u = q$, então a subárvore esquerda de p era vazia. Se $h_E(p) = 0$, não tem como $h_E(p) > h_D(p)$.

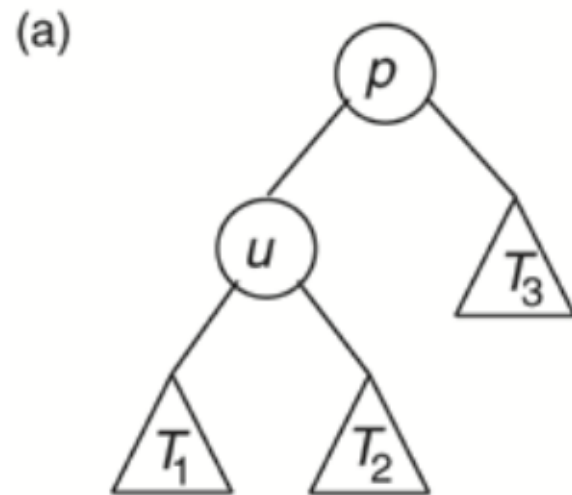
Caso 1: $h_E(p) > h_D(p)$ (3)

- Então, q pertence à subárvore esquerda de p . Além disso, p possui o filho esquerdo $u \neq q$. Pois, caso contrário, p não estaria desbalanceado.

Se $u = q$, então a subárvore esquerda de p era vazia. Então, ou $h_E(p) = h_D(p)$ ou $h_E(p) < h_D(p)$. Em ambos os casos, adicionar um nó q à subárvore esquerda de p não iria tornar a árvore desbalanceado.

Caso 1: $h_E(p) > h_D(p)$ (4)

- Então, q pertence à subárvore esquerda de p . Além disso, p possui o filho esquerdo $u \neq q$. Pois, caso contrário, p não estaria desregulado. Finalmente, por esse mesmo motivo, sabe-se que $h_E(u) = h_D(u)$.



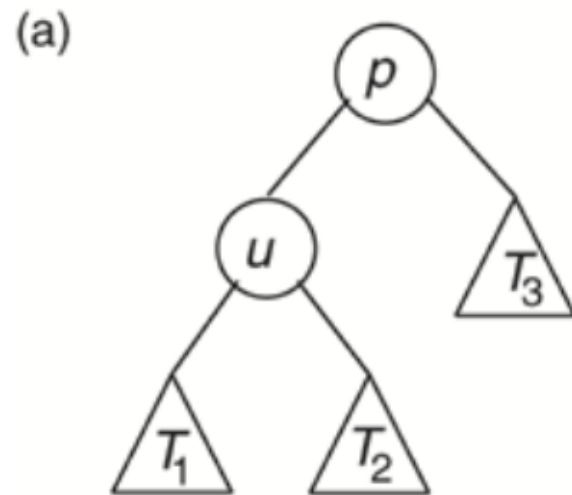
Se $h_E(u) > h_D(u)$, então duas possibilidades podem ocorrer:

- novo nó q ser adicionado à esquerda de u , e portanto u é o nó mais próximo às folhas de T que se tornou desbalanceado, e não p
- novo nó q ser adicionado à direita de u , e nesse caso não aumentaria altura de $h(u)$, e portanto não tornaria o nó p desbalanceado

De forma análoga temos o caso $h_E(u) < h_D(u)$.
Portanto, sabemos que $h_E(u) = h_D(u)$.

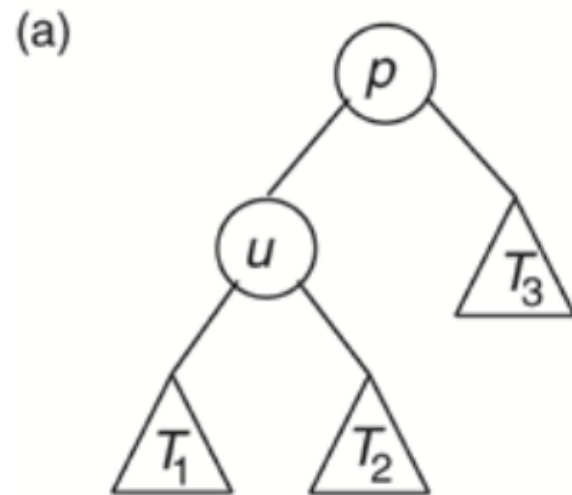
Caso 1.1: $h_E(u) > h_D(u)$

- Essa situação corresponde à da Figura (a), sendo q um nó pertencente a T_1 .



Caso 1.1: $h_E(u) > h_D(u)$ (2)

- Essa situação corresponde à da Figura (a), sendo q um nó pertencente a T_1 . Observe que $h(T_1) - h(T_2) = 1$ e $h(T_2) = h(T_3)$.

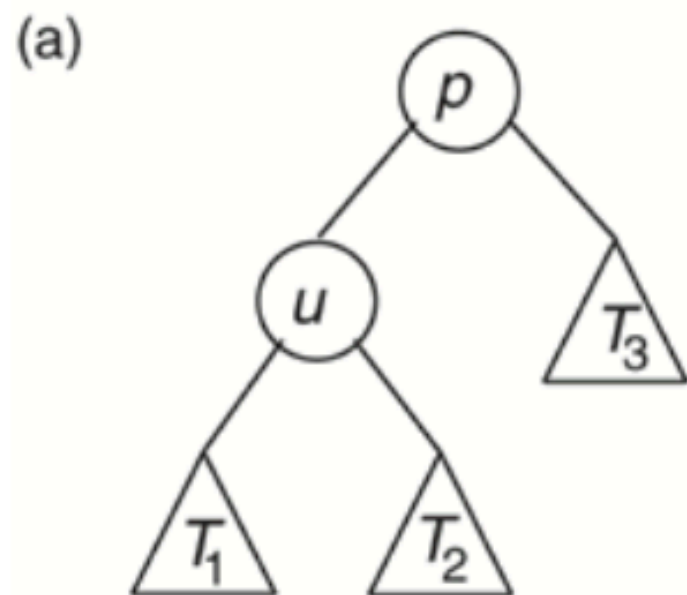


Antes da adição de q , tínhamos $h(T_1) - h(T_2) = 0$. Com um novo nó em T_1 , temos que $h(T_1)$ foi acrescido de 1.

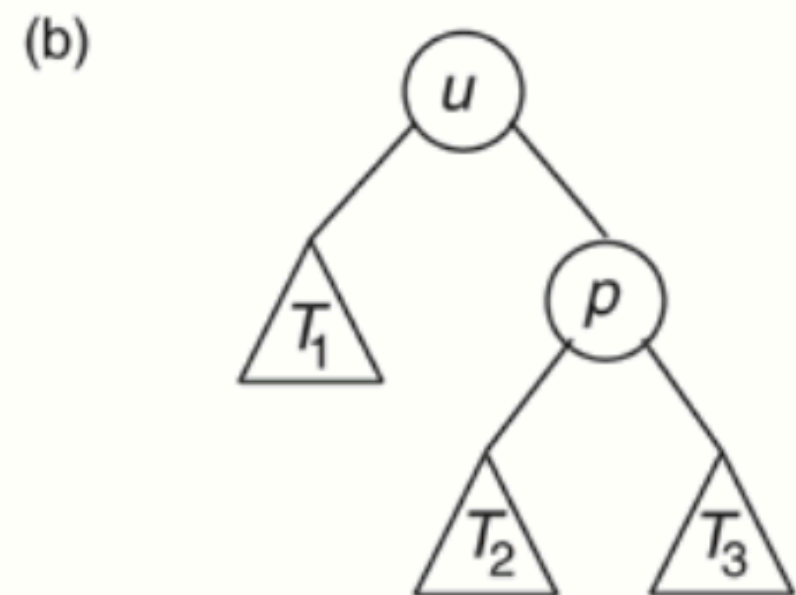
Lembre-se que $h_E(p) > h_D(p)$. Assim, anteriormente tínhamos $h(T_1) = h(T_2) = h(T_3)$, pois caso contrário a árvore não seria balanceada.

Caso 1.1: $h_E(u) > h_D(u)$ (3)

- Essa situação corresponde à da Figura (a), sendo q um nó pertencente a T_1 . Observe que $h(T_1) - h(T_2) = 1$ e $h(T_2) = h(T_3)$. Consequentemente, a aplicação da **rotação direita da raiz p** transforma a subárvore considerada na da Figura (b), o que restabelece a regulagem de p .

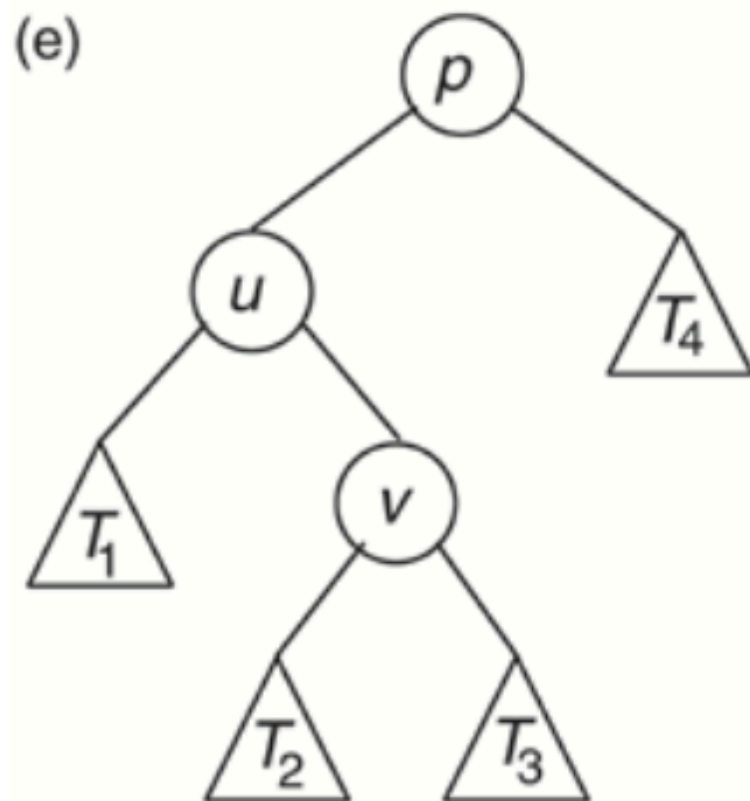


→
rotação
direita



Caso 1.2: $h_D(u) > h_E(u)$

- Então u possui o filho direito v , e a situação é ilustrada na Figura (e).

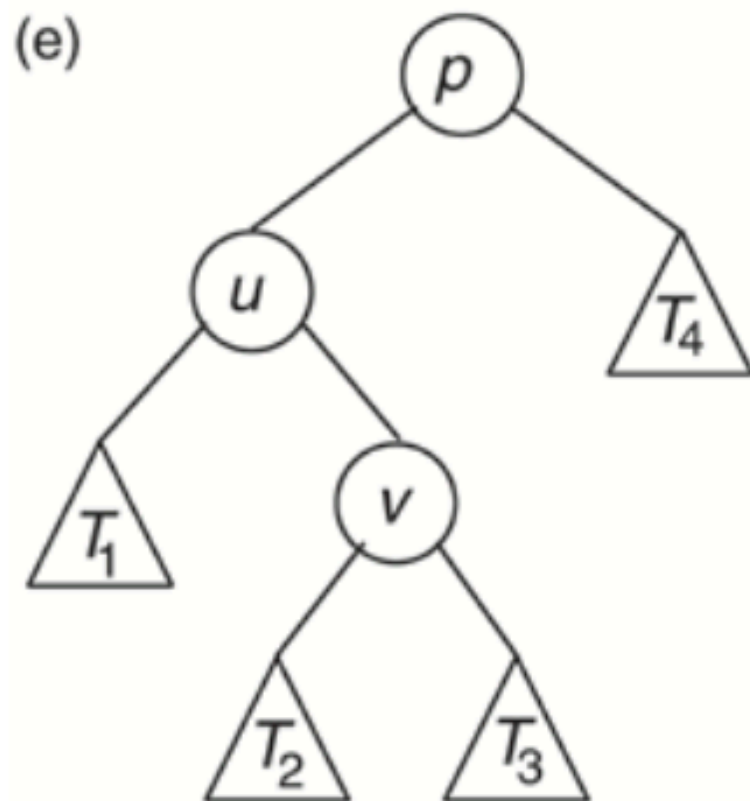


Suponha que u não possua filho direito. Então sua subárvore direita é vazia, implicando $h_D(u) \leq h_E(u)$. Isso contradiz a hipótese $h_D(u) > h_E(u)$. Logo, u possui filho direito v .

Caso 1.2: $h_D(u) > h_E(u)$ (2)

- Então u possui o filho direito v , e a situação é ilustrada na Figura (e). Nesse caso, vale

$$|h(T_2) - h(T_3)| \leq 1 \text{ e} \\ \max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$$

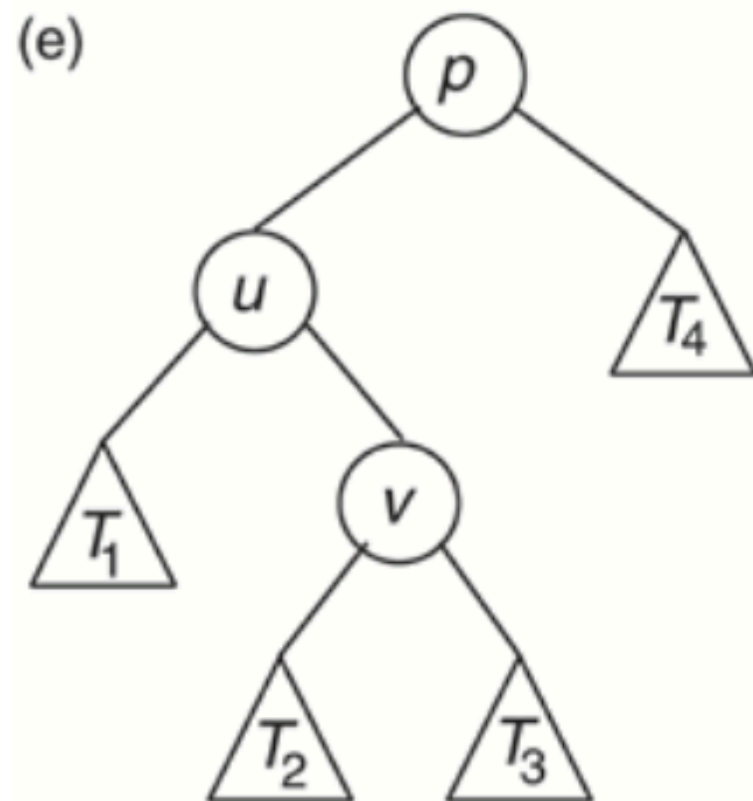


Sabemos que anteriormente $h_E(u) = h_D(u)$. Portanto, $h(v) = h(T_1)$. Ao adicionar um nó q à direita de u , este pode ser inserido em T_2 ou T_3 . Em ambos os casos, $|h(T_2) - h(T_3)| \leq 1$, pois caso contrário o nó mais próximo às folhas de T que se tornou desbalanceado seria v e não p .

Caso 1.2: $h_D(u) > h_E(u)$ (3)

- Então u possui o filho direito v , e a situação é ilustrada na Figura (e). Nesse caso, vale

$$|h(T_2) - h(T_3)| \leq 1 \text{ e} \\ \max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$$



Anteriormente, $h(v) = h(T_1)$. Portanto, o maior dentre $h(T_2)$ e $h(T_3)$ era igual à $h(T_1) - 1$. Para tornar p desbalanceado, o nó q foi adicionado a maior subárvore, seja ela T_2 ou T_3 , caso contrário $h(v)$ não seria alterado e a árvore continuaria balanceada.

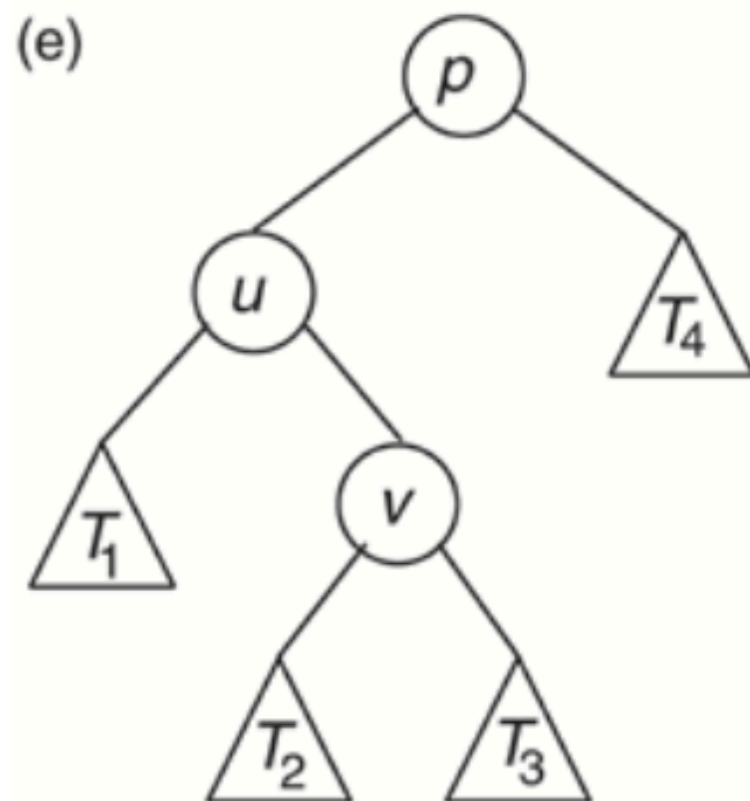
Assim, $\max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$

Caso 1.2: $h_D(u) > h_E(u)$ (4)

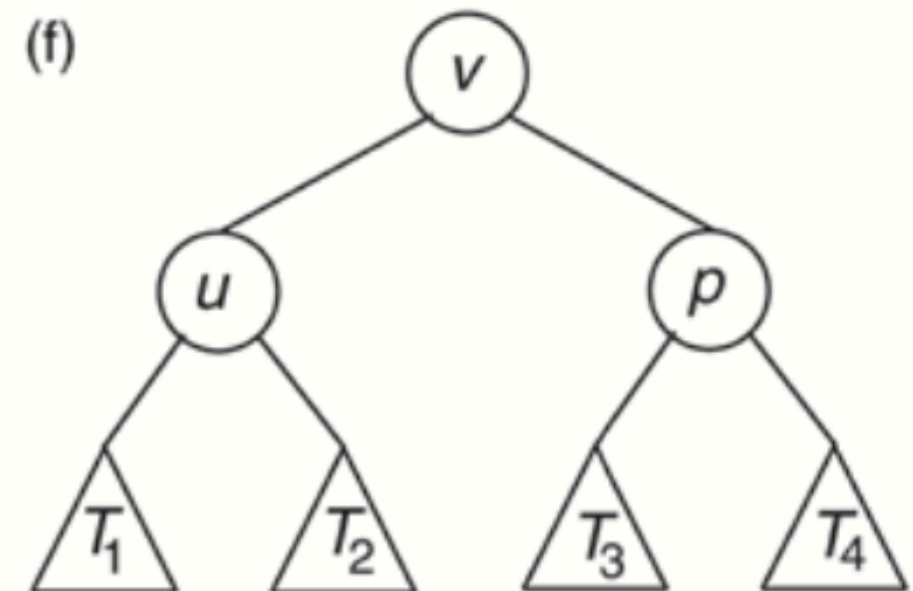
- Então u possui o filho direito v , e a situação é ilustrada na Figura (e). Nesse caso, vale

$$|h(T_2) - h(T_3)| \leq 1 \text{ e} \\ \max\{h(T_2), h(T_3)\} = h(T_1) = h(T_4)$$

- Aplica-se então a **rotação dupla direita da raiz p**



→
rotação
dupla
direita



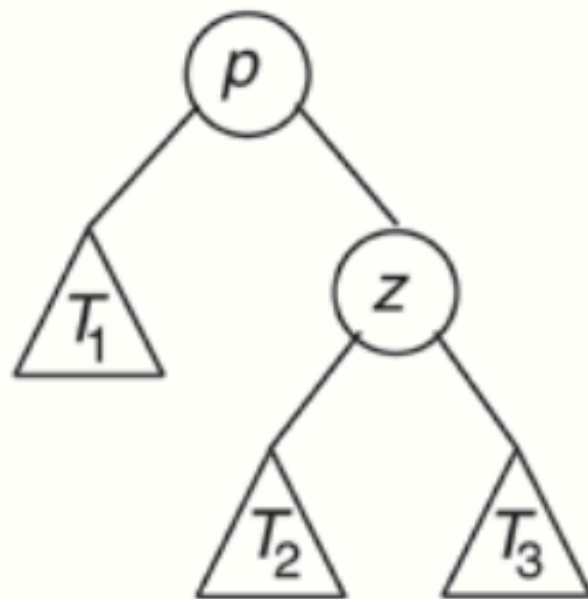
Caso 2: $h_D(p) > h_E(p)$

- Análogo ao caso 1.

Caso 2.1: $h_D(z) > h_E(z)$

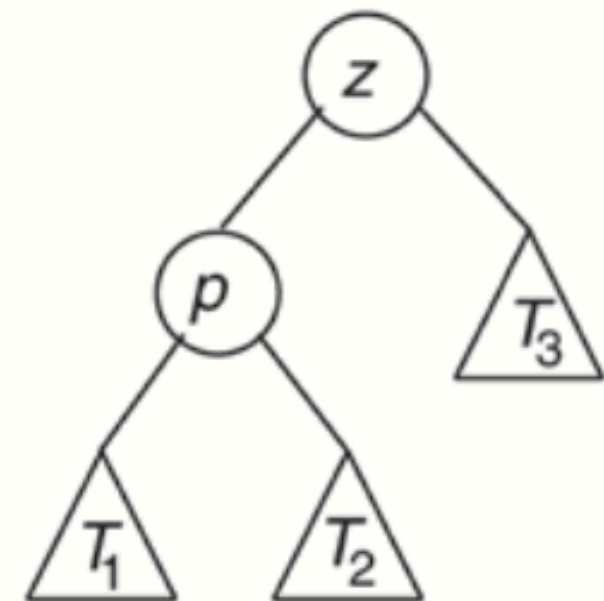
- Análogo ao caso 1.1.

(c)



→
rotação
esquerda

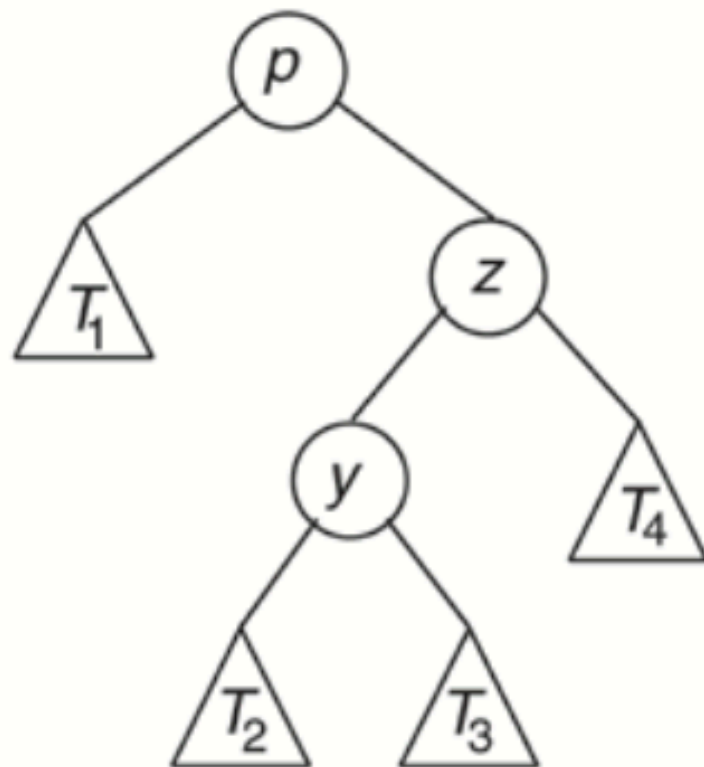
(d)



Caso 2.2: $h_E(z) > h_D(z)$

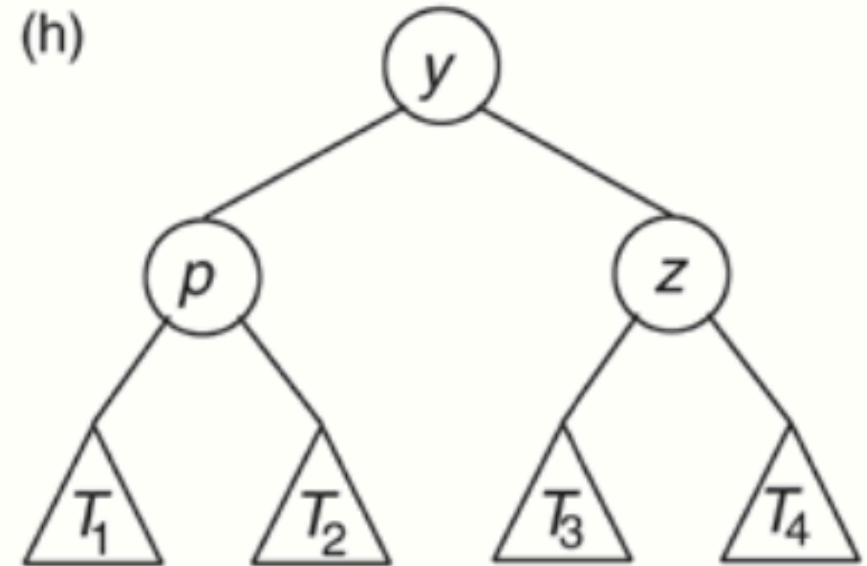
- Análogo ao caso 1.2.

(g)



→
rotação
dupla
esquerda

(h)



Referências

- CORMEN, T. H..[et al]. Algoritmos: teoria e prática. 3^a ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010